



Image by Jiri Holmally from Pixabay

Partitioned Matchmaking Service

Salam Zahreddine | Engelbrecht Jerome | Yuting Feng | July 13, 2026

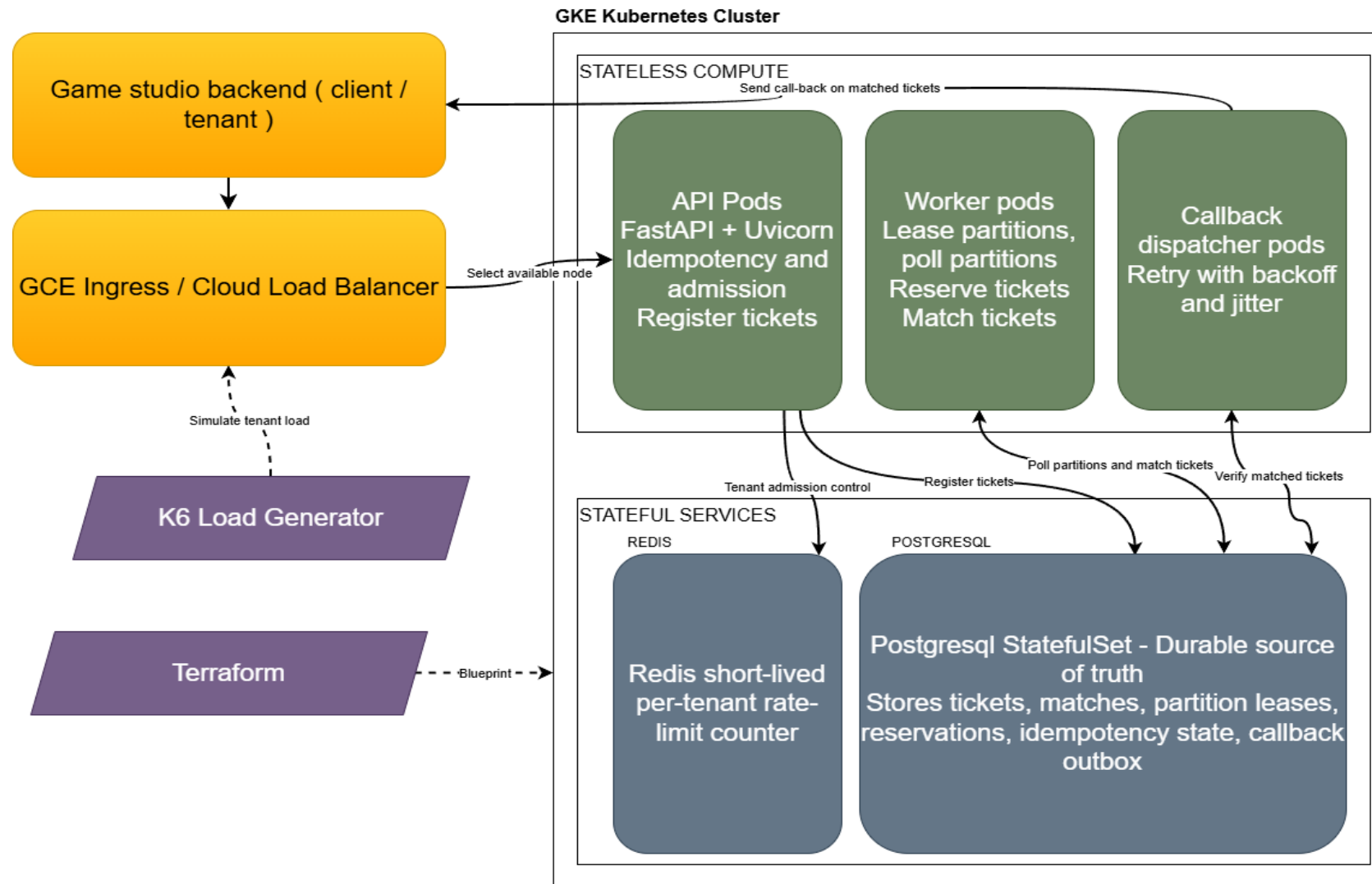
Application Overview

- Game studios (tenants) submit tickets for players, the service pairs compatible players into matches based on tenant, region, queue type, and skill rating.
- Results get delivered via signed callbacks instead of polling

Why is it interesting?

- Multi-tenant fairness
- High write throughput
- Stateful matching with fault recovery
- All under configurable overload protection

Architecture



- Admission Control :
- Tenant rate limit
 - Tenant waiting quota
 - Partition backlog limit
 - DB-Latency load shedding

Horizontal Scaling

Config	Accepted Tickets	Match Throughput	API p95 Latency
1 node	58.9/s	6.53/s	500ms
3 nodes	71.3/s	8.53/s	70ms
5 nodes	68.2/s	33.05/s	38ms

Key findings:

- Ticket acceptance is rate-limit bound (~55/s aggregate cap), flat across node counts as designed
- Match throughput is the useful scale-out signal, +287% from 3 to 5 nodes for +67% more workers, superlinear due to better partition lease coverage
- API latency confirms stateless horizontal scaling

Vertical Scaling

Config	matches/s (E2 → N2)	p95 latency (E2 → N2)
1 node	6.5 → 6.3	500ms → 196ms
3 nodes	8.5 → 28.8	70ms → 34ms
5 nodes	33.1 → 29.2	38ms → 34ms

Key findings:

- Faster per-core performance mainly helps API latency at 1 node and worker throughput at 3 nodes
- both machine families plateau near ~30 matches/s at 5 nodes.

Limitations

- Partition fragmentation (many thin queues across tenants/regions) can depress pairing efficiency until enough worker replicas cover distinct partitions.
- Single point of failure on for the database
- Built for 1 to 1 tickets

Partitioned Matchmaking Service

Conclusion